Compito 9.1

Malchiodi Riccardo 5680500

Obiettivo del Laboratorio

L'obiettivo di questo compito è implementare e analizzare l'algoritmo **MCMatrixMultiplicationVerifier**, un metodo Monte Carlo probabilistico utilizzato per verificare se il prodotto di due matrici A e B è uguale a una matrice C.

Nello specifico, l'esperimento si propone di:

1. Generare due matrici casuali A e B di dimensione 100×100.

2. Costruire una matrice risultato C contenente un **errore intenzionale**: l'aggiunta di +1 a un elemento e la sottrazione di −1 a un altro elemento della **stessa riga**.

3. Eseguire l'algoritmo di verifica per un numero variabile di iterazioni (k=2,5,10) ripetendo il test 100 volte.

4. Dimostrare empiricamente che la probabilità che l'algoritmo fallisca (ovvero che dia un "Falso Positivo" dicendo che C è corretta) diminuisce esponenzialmente all'aumentare di k, seguendo la legge teorica $P(errore) \leq 1/2^k$.

Analisi Output

```
Inserimento errore in riga 50: colonna 20 (+1) e colonna 80 (-1)

--- INIZIO TEST MONTE CARLO ---

Risultati per k = 2:
  - Errori rilevati (successo): 75 su 100
  - Falsi positivi (fallimento): 25 su 100
  - Probabilità teorica di fallimento: ~25.0000%

Risultati per k = 5:
  - Errori rilevati (successo): 96 su 100
  - Falsi positivi (fallimento): 4 su 100
  - Probabilità teorica di fallimento: ~3.1250%

Risultati per k = 10:
  - Errori rilevati (successo): 100 su 100
  - Falsi positivi (fallimento): 0 su 100
  - Probabilità teorica di fallimento: ~0.0977%
```

1. Caso k = 2
    - **Risultati empirici:** l'algoritmo ha rilevato l'errore in 75 casi, ma ha generato un falso positivo in 25 casi su 100.
    - **Confronto teorico:** il tasso di fallimento osservato del 25% coincide esattamente con la probabilità teorica di errore $P(errore) \leq 1/2^2 = 1/4 = 25\%$
    - Con sole 2 iterazioni, l'algoritmo è veloce ma ancora inaffidabile.

2. Caso k = 5
    - **Risultati empirici:** aumentando le iterazioni a 5, i falsi positivi sono scesi drasticamente a 4 su 100
    - **Confronto teorico:** il tasso di fallimento del 4%
    - Con un numero ancora contenuto di iterazioni, l'affidabilità migliora sensibilmente, riducendo l'errore di oltre sei volte rispetto al caso k = 2

3. Caso k = 10
    - **Risultati empirici:** l'algoritmo ha raggiunto la perfezione rilevando l'errore in 100 casi su 100 con 0 falsi positivi
    - **Confronto teorico:** la probabilità teorica di errore è trascurabile, pari a circa 0.097% $(1/2^{10})$
    - Questo conferma che k = 10 è sufficiente per garantire un'altissima affidabilità pratica, rendendo quasi impossibile il verificarsi di un falso positivo

Conclusioni

Sulla base dei dati sperimentali raccolti e dell'analisi teorica, possiamo trarre le seguenti conclusioni sull'algoritmo **MCMatrixMultiplicationVerifier**:

- **Efficienza Computazionale:** L'algoritmo si conferma un metodo efficiente con complessità $O(n^2)$ ideale per la verifica rapida di prodotti matriciali rispetto ai metodi deterministici classici.

- **Decadimento Esponenziale dell'Errore:** Gli esperimenti hanno confermato la legge teorica secondo cui la probabilità di errore diminuisce esponenzialmente all'aumentare delle iterazioni, seguendo la formula $P(errore) \leq (1/2)^k$

- **Affidabilità:** È stato dimostrato che non sono necessarie grandi quantità di iterazioni per ottenere risultati sicuri. Anche con un piccolo numero di iterazioni (es. k=10), si ottiene un'altissima affidabilità.

Codice

```
import numpy as np

def mc_matrix_multiplication_verifier(A, B, C, k):

    n = A.shape[0]

    # Ripetiamo il test k volte
    for _ in range(k):
        r = np.random.randint(0, 2, size=(n, 1))

        # Lato Destro: C * r
        u = C @ r

        # Lato Sinistro: A * (B * r)
        s = B @ r
        t = A @ s

        # Se t != u, abbiamo trovato sicuramente un errore
        if not np.array_equal(t, u):
            return False # AB != C confermato

    return True # Probabilmente AB == C



n = 100
possibili_valori = [-2, -1, 0, 1, 2]
A = np.random.choice(possibili_valori, size=(n, n))
B = np.random.choice(possibili_valori, size=(n, n))

C_corretta = A @ B

C_corrotta = C_corretta.copy()
riga_target = 50  # Scegliamo una riga a caso, es. la 50
col_1 = 20        # Colonna 20
col_2 = 80        # Colonna 80

print(f"Inserimento errore in riga {riga_target}: colonna {col_1} (+1) e colonna
{col_2} (-1)")
```

```python
C_corrotta[riga_target, col_1] += 1
C_corrotta[riga_target, col_2] -= 1

valori_k = [2, 5, 10]
totale_prove = 100

print("\n--- INIZIO TEST MONTE CARLO ---")

for k in valori_k:
   errori_rilevati = 0
   falsi_positivi = 0 # Quante volte dice "Corretto" anche se è sbagliato

   for _ in range(totale_prove):
       risultato = mc_matrix_multiplication_verifier(A, B, C_corrotta, k)

       if risultato == False:
           errori_rilevati += 1
       else:
           falsi_positivi += 1

   print(f"\nRisultati per k = {k}:")
   print(f"  - Errori rilevati (successo): {errori_rilevati} su {totale_prove}")
   print(f"  - Falsi positivi (fallimento): {falsi_positivi} su {totale_prove}")
   theoretical_prob = (1/(2**k)) * 100
   print(f"  - Probabilità teorica di fallimento: ~{theoretical_prob:.4f}%")
```